Homework 1
Overfitting and Decision Trees[1]

CMU 10-701: Introduction to Machine Learning (Fall 2026)

https://piazza.com/cmu/fall2026/10701/home
OUT: Friday, Aug. 28, 2026

DUE: Tuesday, Sep. 8, 2026, 11:59pm

START HERE: Instructions

- **Collaboration policy:** Collaboration on solving the homework is allowed, after you have thought about the problems on your own. It is also OK to get clarification (but not solutions) from books or online resources, again after you have thought about the problems on your own. There are two requirements: first, cite your collaborators fully and completely (e.g., "Jane explained to me what is asked in Question 2.1"). Second, write your solution *independently*: close the book and all of your notes, and send collaborators out of the room, so that the solution comes from you only. See the Academic Integrity Section in our Course Syllabus for more information.
- **Late Submission Policy:** See the late homework policy in our Course Syllabus
- **Submitting your work:**
  - **Gradescope:** There will be two submission slots for this homework on Gradescope: Written and Programming.
    For the written problems such as short answer, multiple choice, derivations, proofs, or plots, we will be using the written submission slot. Please use the provided template. The best way to format your homework is by using the Latex template released in the handout and writing your solutions in Latex. However submissions can be handwritten onto the template, but should be labeled and clearly legible. If your writing is not legible, you will not be awarded marks. Each derivation/proof should be completed in the boxes provided below the question, **you should not move or change the sizes of these boxes** as Gradescope is expecting your solved homework PDF to match the template on Gradescope. If you find you need more space than the box provides you should consider cutting your solution down to its relevant parts, if you see no way to do this, please add an additional page at the end of the homework and guide us there with a 'See page xx for the rest of the solution'.
    You are also required to upload your code, which you wrote to solve the final question of this homework, to the Programming submission slot. Your code may be run by TAs so please make sure it is in a workable state.
    Regrade requests can be made after the homework grades are released, however this gives the TA the opportunity to regrade your entire paper, meaning if additional mistakes are found then points will be deducted.

For multiple choice or select all that apply questions, shade in the box or circle in the template document corresponding to the correct answer(s) for each of the questions. For LaTeXusers, use ■ and ●for shaded boxes and circles, and don't change anything else. If an answer box is included for showing work, **you must show your work**!

[1]Compiled on Saturday 29th August, 2026 at 02:21

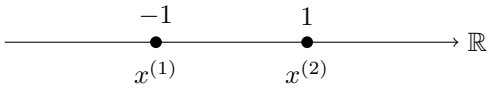
1 Overfitting in 1D

Scotty just learned how a linear binary classifier for $\mathbb{R}$ works. The binary labels are $\{-, +\}$. A linear classifier $C : \mathbb{R} \to \{-, +\}$ determines the label $C(x)$ associated with any $x \in \mathbb{R}$. In $\mathbb{R}$, a classifier is a linear classifier if and only if it can be formulated as follows:

$$C(x) = \begin{cases} + \text{ if } wx + b \geq 0 \\ - \text{ else} \end{cases}$$

where $w, b \in \mathbb{R}$ are fixed.

Scotty is currently on vacation in a 1d world. He walks along a path in $\mathbb{R}$. There are two points $x^{(1)}, x^{(2)}$ at positions -1 and 1 respectively.



Scotty has a metal detector and is trying to see if there is treasure at positions $x^{(1)}, x^{(2)}$.
A label of "$-$" indicates no treasure, and "$+$" indicates treasure. In the true, unknown state of the world, there is gold under $x^{(2)}$ but no gold at $x^{(1)}$. The true label for $x^{(1)}$ is "$-$" and for $x^{(2)}$ is "$+$".

Scotty trains a linear model to determine where there is treasure or not. However, there is noise and the metal detector is not always accurate. Let $\alpha \in [0, 1]$ be the noise factor. The metal detector returns the wrong label with probability $\frac{\alpha}{2}$, independently for each measurement. (So, we can think of α as the probability that we replace the true label by uninformative (uniform) noise.) The probabilities are shown in the following table:

Table 1: Metal detector result probability

	$x^{(1)}$	$x^{(2)}$
$-$	$1 - \frac{\alpha}{2}$	$\frac{\alpha}{2}$
$+$	$\frac{\alpha}{2}$	$1 - \frac{\alpha}{2}$

Scotty wants to use 0-1 loss: the penalty of incorrectly predicting a value is 1; in other words:

$$L(y, \hat{y}) = \mathbb{1}[y \neq \hat{y}]$$

Scotty gets two (statistically independent) readings, one from $x^{(1)}$ and one from $x^{(2)}$, which he uses as training data. He uses the training data to create a classifier that minimises the loss (with respect to training data).

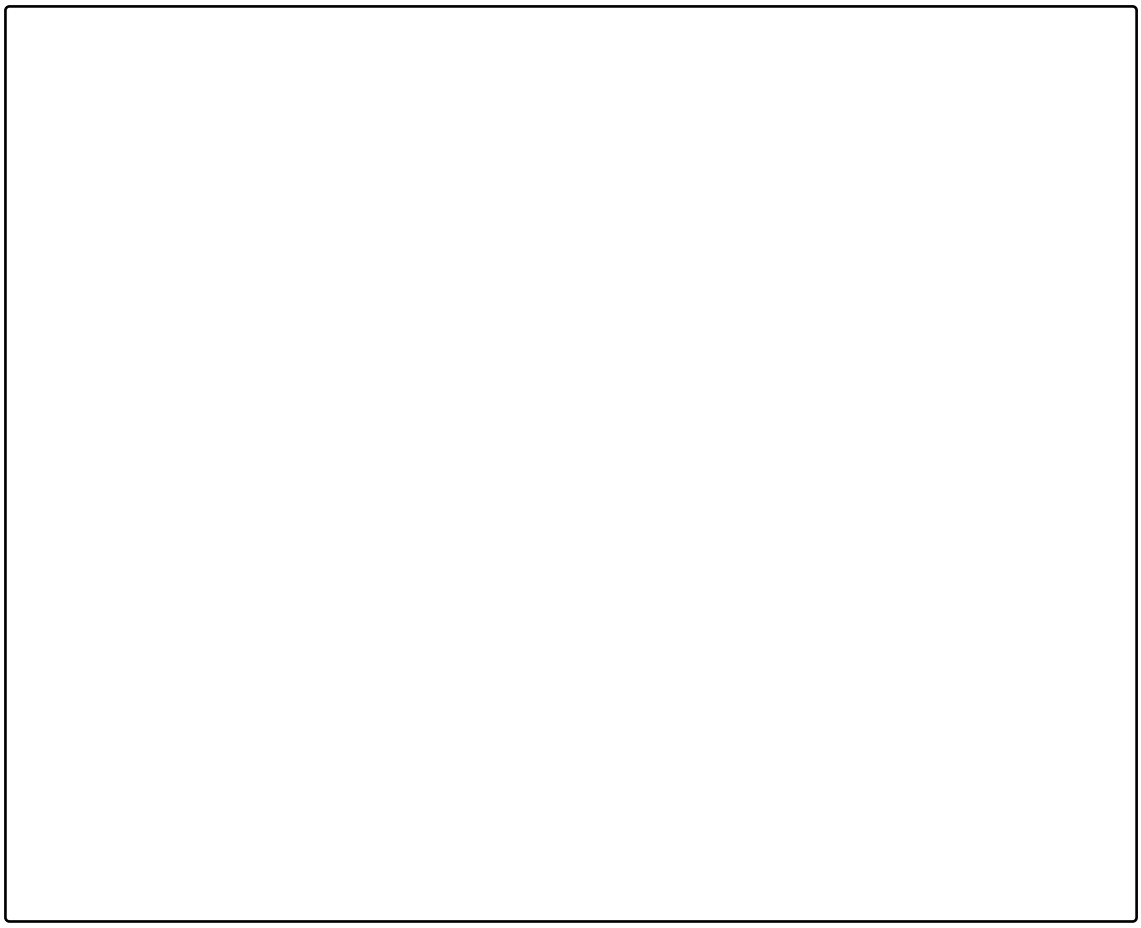
1. **[2 points]** Training means picking a linear binary classifier that scores best on the **training** data. Using only two points $x^{(1)}, x^{(2)}$, for training, there are only 4 unique classifiers that Scotty can pick. We will treat classifiers that induce the same labels on the same points as the same classifier. The outputs are shown the the rows of the table below. Fill in the probability that Scotty will pick each of these classifiers (with respect to the randomness in the training data). Leave answer in terms of α.

Label of $x^{(1)}$	Label of $x^{(2)}$	Probability
$+$	$+$	
$+$	$-$	
$-$	$+$	
$-$	$-$	

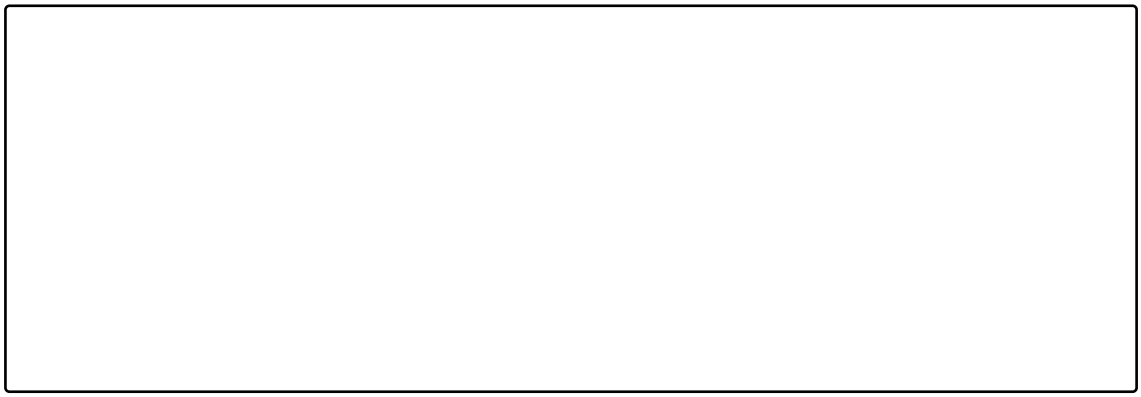
2. **[2 points]** To generate a test set, Scotty collects another observation randomly with equal probability from $x^{(1)}$ or $x^{(2)}$. The metal detector's probability outputs remain the same as before.
   For each of the four classifiers, compute the expected error on this new example (i.e., the expected test error). Leave the answer as a function of α.

3. **[2 points]** What is the expected test error averaged over the randomness in the training set? Give your answer in terms of α.

4. **[4 points]** Create a plot that shows three lines:
   - the expected test error at $\alpha \in \{0.0, 0.25, 0.5, 0.75, 1.0\}$
   - the minimum possible test error at these same values of α — in other words, the error if Scotty knew the correct classifier ahead of time, and evaluated it on a newly sampled test example.
   - the expected training error at these same values of α.



5. **[2 points]** The difference between the first and second lines in **part 4** is one possible measure of the amount of overfitting: the expected excess test error because our training process has to work from a limited-size, noisy training set. Similarly, the difference between the first and third lines in **part 4** is another measure: the expected optimism when comparing training to test error. At what α value is overfitting the worst according to each of these measures? Why?



2 Repeating the Overfitting Analysis in Two Dimensions

Scotty now visits a two-dimensional world. For a point $x = (x_1, x_2)$, an affine linear classifier has the form

$$C_{\mathbf{w},b}(x) = \begin{cases} +, & w_1 x_1 + w_2 x_2 + b \geq 0, \\ -, & w_1 x_1 + w_2 x_2 + b < 0, \end{cases}$$

where $\mathbf{w} = (w_1, w_2) \neq (0, 0)$ and $b \in \mathbb{R}$.

Consider the following two sets of points, where S_3 consists of three vertices of a square and S_4 consists of all four vertices:

$$S_3 = \{x^{(1)} = (-1, -1),\ x^{(2)} = (-1, 1),\ x^{(3)} = (1, 1)\},$$
$$S_4 = \{x^{(1)} = (-1, -1),\ x^{(2)} = (-1, 1),\ x^{(3)} = (1, 1),\ x^{(4)} = (1, -1)\}.$$

The noise-free labels are determined by the vertical linear classifier

$$f^*(x) = \begin{cases} -, & x_1 < 0, \\ +, & x_1 > 0. \end{cases}$$

Thus, in the point order displayed above, the noise-free label vectors are $(-, -, +)$ for S_3 and $(-, -, +, +)$ for S_4.

As in Question 1, Scotty observes one noisy training label at every point. For each point independently, with probability $\alpha \in [0, 1]$, the noise process *replaces* its noise-free label with the result of a fair coin flip; with probability $1 - \alpha$, the noise-free label is retained. Therefore

$$\Pr(\text{observed label is correct}) = 1 - \frac{\alpha}{2}, \qquad \Pr(\text{observed label is incorrect}) = \frac{\alpha}{2}.$$

In particular, when $\alpha = 1$, the observed label is equally likely to be $+$ or $-$, regardless of the noise-free label. For convenience, define

$$q = \frac{\alpha}{2}.$$

Scotty chooses an affine linear classifier that minimizes its average 0–1 loss on the training set. Again, classifiers that induce the same labels on the finite point set are treated as one classifier. If several distinct induced labelings attain the same minimum training error, Scotty chooses uniformly at random among them. This tie-breaking rule is important for S_4.

To obtain a test example, first choose a point uniformly from S_m and then obtain a new, independent label using the same replacement-noise process and the same value of α. In particular, the test label is independent of the training label conditional on the selected point.

(a) **[2 points]** For each of S_3 and S_4, how many of the possible binary labelings are realizable by an affine linear classifier? Identify any non-realizable labelings and explain geometrically why a straight line cannot separate their positive and negative points.

(b) **[2 points]** Compute the probability that Scotty learns each possible classifier on S_3 and S_4. To avoid listing every classifier separately, group learned labelings by their Hamming distance k from the noise-free label vector. In your table, report (i) the number of learned labelings in each group and (ii) the probability of learning *each individual labeling* in that group. Be careful to distinguish the probability of *observing* a particular training labeling from the probability of *learning* a classifier that induces that labeling on the point set.

(c) **[2 points]** Suppose a learned classifier disagrees with the noise-free label vector at exactly k of the m points in S_m. Compute its expected test error as a function of k, m, and α.

(d) **[2 points]** Average over the randomness in the training data and Scotty's tie-breaking. For both S_3 and S_4, compute (i) the expected training error and (ii) the expected test error.

(e) **[4 points]** Make two plots, one for S_3 and one for S_4. On each plot, show:

- the expected test error at $\alpha \in \{0, 0.25, 0.5, 0.75, 1\}$.
- the minimum possible expected test error at each of these α values for an oracle that knows f^*.
- the expected training error at each α.

Where is overfitting the worst, according to the difference between the first and second lines, and the difference between the first and third lines?

(f) **[3 points]** You probably noticed that, on S_4, Scotty sometimes obtains strictly positive training error, unlike the earlier setups. Qualitatively, how does the analysis change if we replace affine linear classifiers with a more expressive hypothesis class that can realize all 16 labelings of S_4 and therefore always attain zero training error? What happens to our measures of overfitting?

3 Decision Trees [20 pts]

1. Consider the dataset in Table 2 for this problem. Given the four attributes on the left, we want to predict if the student got an A in the course. The following questions involve some computation, so you may want to solve them programmatically. You can find this dataset in *data.txt*

Wakes Up Early	**Finished All Homework**	**Completed Pre-requisites**	**Likes Coffee**	**A**
1	1	0	0	0
1	1	1	0	1
0	0	1	0	0
0	1	1	0	1
0	1	1	0	1
0	0	1	1	0
1	0	0	0	0
0	1	0	1	1
0	0	1	0	1
1	0	0	0	0
1	1	1	0	1
0	1	1	1	0
0	0	0	0	0
1	0	0	1	0

Table 2: decision tree features and labels

Create a decision tree of depth 2 using the ID3 algorithm from class. The tree should have the same structure as the diagram in Figure 1. Note that 0 leads to the left branch and 1 leads to the right branch.

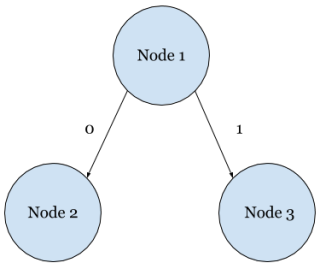


Figure 1: Decision tree structure for question 1

(i) **[2 pts]** What is the attribute at Node 1? What is the information gain of this attribute? Please round your answer to 4 decimal places.

Attribute to split on: [ ] Mutual information: [ ]

(ii) **[2 pts]** What is the attribute at Node 2? What is the information gain of this attribute? Please round your answer to 4 decimal places.

Attribute to split on: [ ] Mutual information: [ ]

(iii) **[2 pts]** What is the attribute at Node 3? What is the information gain of this attribute? Please round your answer to 4 decimal places.

Note: If there is a tie on the attribute with the highest information gain, write any of the attribute that has the highest information gain.

Attribute to split on: [ ] Mutual information: [ ]

2. **[4 pts]** Consider learning a decision tree for some binary classification task. Assume that we do not restrict the size of the tree and that the tree can branch multiple times on the same attribute. Under what condition(s) would we be unable to construct a tree that perfectly classifies the dataset?

3. **[4 pts]** Assuming that there are no inconsistencies in the data, is the ID3 algorithm guaranteed to output the smallest decision tree that can perfectly classify the training data i.e., achieves zero training error rate? Briefly explain why or why not.
Note: Inconsistent data is when there are at least two data points $x^{(i)}$ and $x^{(j)}$ with identical feature values, such that $y^{(i)} \neq y^{(j)}$.

4. **[6 Points]** Recall that the information gain of some variable A is the expected reduction in entropy of a target variable Y in some dataset S due to partitioning on A. Prove that information gain is always non-negative. To receive full credit, you must show all steps of your work. **Hint:** You may find the result proved in Recitation 1 regarding the *KL divergence* between two distributions useful for this question.

4 Programming (55 points)

In this assignment, you will implement a decision tree learning algorithm for binary classification. In addition, we will ask you to run some end-to-end experiments on two tasks (predicting whether or not a patient has heart disease / predicting the final grade for high school students) and report your results. Please confine all code you write to a single, self-contained file titled `decision_tree.py`.

4.1 The Tasks and Datasets

Materials Download the zip file from the course website. The zip file will have a handout folder that contains all the data that you will need in order to complete this assignment.

Datasets The handout contains three datasets. Each one contains attributes and labels and is already split into training and validation data. The first line of each `.tsv` file contains the name of each attribute, and *the class label is always the last column.*

1. **heart:** The first task is to predict whether a patient has been (or will be) diagnosed with heart disease, based on available patient information. The attributes (aka. features) are:
   (a) `sex`: The sex of the patient—1 if the patient is male, and 0 if the patient is female.
   (b) `chest_pain`: 1 if the patient has chest pain, and 0 otherwise.
   (c) `high_blood_sugar`: 1 if the patient has high blood sugar (>120 mg/dl fasting), and 0 otherwise.
   (d) `abnormal_ecg`: 1 if the patient had an abnormal resting electrocardiographic (ECG) reading, and 0 otherwise.
   (e) `angina`: 1 if exercise induced angina in the patient, and 0 otherwise. Angina is a type of severe chest pain.
   (f) `flat_ST`: 1 if the patient's ST segment (a section of an ECG) was flat during exercise, and 0 if it had some slope.
   (g) `fluoroscopy`: 1 if a physician used fluoroscopy, and 0 otherwise. Fluoroscopy is an imaging technique used to see the flow of blood through the heart.
   (h) `thalassemia`: 1 if the patient is known to have thalassemia, and 0 otherwise. Thalassemia is a blood disorder that may impair the oxygen-carrying capacity of the patient's red blood cells.
   (i) `heart_disease`: 1 if the patient was diagnosed with heart disease, and 0 otherwise. This is the class label you should predict.

   The training data is in `heart_train.tsv`, and the validation data in `heart_val.tsv`.
2. **education:** The second task is to predict the final grade for high school students. The attributes are student grades on 5 multiple choice assignments *M1* through *M5*, 4 programming assignments *P1* through *P4*, and the final exam *F*. Values of 1 indicate that a student received an A, and 0 indicates that the student did not receive an A. The training data is in `education_train.tsv`, and the validation data in `education_val.tsv`.
3. **small:** We also include `small_train.tsv` and `small_val.tsv`—a small, purely for demonstration version of the **heart** dataset, with *only* attributes `chest_pain` and `thalassemia`.

4.2 Program: Decision Tree

Your code should learn a decision tree with a specified maximum depth, print the decision tree in a specified format, predict the labels of the training and validation examples, and calculate training and validation errors.

Your implementation must satisfy the following requirements:

- Use mutual information to determine which attribute to split on.
- Be sure you're correctly weighting your calculation of mutual information. For a split on attribute X, $I(Y;X) = H(Y) - H(Y|X) = H(Y) - P(X=0)H(Y|X=0) - P(X=1)H(Y|X=1)$.
- As a stopping rule, only split on an attribute if the mutual information is $>$ a certain threshold. By default, you should set the threshold to 0. Certain questions in 4.3 may require you to experiment with different thresholds.
- Do not grow the tree beyond a certain max-depth, if specified in the question. For example, for a maximum depth of 3, split a node only if the mutual information is > 0 and the current level of the node is < 3.
- Use a majority vote of the labels at each leaf to make classification decisions. If the vote is tied, choose the label that is numerically larger (i.e., 1 should be chosen before 0)
- It is possible for different columns to have equal values for mutual information. In this case, you should split on the ***first column to break ties*** (e.g. if column 0 and column 4 have the same mutual information, use column 0).

4.2.1 Getting Started

Careful planning will help you to correctly and concisely implement your decision tree learning algorithm. Here are a few *hints* to get you started:

- Write helper functions to calculate entropy and mutual information.
- It is best to think of a decision tree as a collection of nodes, where nodes are either leaf nodes (where final decisions are made) or interior nodes (where we split on attributes). It is helpful to design a function to train a single node (i.e. a depth-0 tree), and then recursively call that function to create sub-trees.
- In the recursion, keep track of the depth of the current tree so you can stop growing the tree beyond the max-depth.
- Implement a function that takes a learned decision tree and data as inputs, and generates predicted labels. You can write a separate function to calculate the error of the predicted labels with respect to the given (ground-truth) labels.
- Be sure to correctly handle the case where the specified maximum depth is greater than the total number of attributes.
- Be sure to handle the case where max-depth is zero (i.e. a majority vote classifier).

4.2.2 Output: Printing the Tree to Text Files

You should also write a function to pretty-print your learned decision tree. **Your function should print your tree only *after* you are done generating the fully-trained tree.** Each row should correspond to a node in the tree. They should be printed using a *pre-order depth-first-search* traversal (you should print left-to-right or right-to-left, i.e. your answer should have exactly the same order as the reference below). Print the attribute of the node's parent and the attribute value corresponding to the node. Also include the sufficient statistics (i.e. count of positive / negative examples) for the data passed to that node. The row for the root should include *only* those sufficient statistics. A node at depth d, should be prefixed by d copies of the string '| '.

Below, we have provided the correct format for printing the tree. You should print it to a file named in a specific convention (detailed later in relevant questions). Each separate line should end with a newline character '\n', including the last leaf of the tree.

```
$ python decision_tree.py small_train.tsv small_val.tsv 2

[14 0/14 1]
| chest_pain = 0: [4 0/12 1]
| | thalassemia = 0: [3 0/4 1]
| | thalassemia = 1: [1 0/8 1]
| chest_pain = 1: [10 0/2 1]
| | thalassemia = 0: [7 0/0 1]
| | thalassemia = 1: [3 0/2 1]
```

However, you should be careful that the tree might not be full. For example, with a different subset of the small dataset, there may be no nodes under `chest_pain = 0` if all labels are the same.

The following pretty-print shows the education dataset with max-depth 3. Use this example to check your code before submitting your pretty-print of the heart dataset.

```
$ python decision_tree.py education_train.tsv education_val.tsv 3

[65 0/135 1]
| F = 0: [42 0/16 1]
| | M2 = 0: [27 0/3 1]
| | | M4 = 0: [22 0/0 1]
| | | M4 = 1: [5 0/3 1]
| | M2 = 1: [15 0/13 1]
| | | M4 = 0: [14 0/7 1]
| | | M4 = 1: [1 0/6 1]
| F = 1: [23 0/119 1]
| | M4 = 0: [21 0/63 1]
| | | M2 = 0: [18 0/26 1]
| | | M2 = 1: [3 0/37 1]
| | M4 = 1: [2 0/56 1]
| | | P1 = 0: [2 0/15 1]
| | | P1 = 1: [0 0/41 1]
```

The numbers in brackets give the number of positive and negative labels from the training data in that part of the tree.

At this point, you should be able to answer questions 1-6 in the "Empirical Questions" of this handout. Write your solutions in the template provided.

4.3 Written: Empirical Questions

1. **[5 Points]** Report the validation accuracy of the decision tree classifiers trained on the three included datasets (with no limitation on maximum depth and all configurations like information gain threshold etc. set to the default value):

   - `small` dataset:
   - `heart` dataset:
   - `education` dataset:

2. **[5 Points]** This question will be autograded on Gradescope along with 4.3.3, please submit your `decision_tree.py` file to the Homework 1 Programming slot on Gradescope for both questions to be autograded at the same time. In the box below, paste the decision tree learned on `heart_train.tsv` under max depth limitation of 4. Be sure to follow the formatting requirement in section 4.2.2. Your `decision_tree.py` file should take the following command:

   ```
   $ python decision_tree.py heart_train.tsv heart_val.tsv 4 train
   ```

   and output the file named `trained_tree.txt`.

3. **[10 Points]** This question will be autograded on Gradescope along with 4.3.2, please submit your `decision_tree.py` file to the Homework 1 Programming slot on Gradescope for both questions to be autograded at the same time. Now use `heart_val.tsv` to perform reduced error pruning on the tree you learned in the previous question; break ties in favor of shorter trees. In the box below, paste the decision tree learned after pruning. Be sure to follow the formatting requirement in section 4.2.2.

   Your `decision_tree.py` file should take the following command:

   ```
   $ python decision_tree.py heart_train.tsv heart_val.tsv 4 prune
   ```

   and output the file named `pruned_tree.txt`.

   Hint: After performing reduced error pruning, this is what the education dataset with max-depth 3 looks like:

   ```
   [65 0/135 1]
   | F = 0:  [42 0/16 1]
   | | M2 = 0:  [27 0/3 1]
   | | M2 = 1:  [15 0/13 1]
   | | | M4 = 0:  [14 0/7 1]
   | | | M4 = 1:  [1 0/6 1]
   | F = 1:  [23 0/119 1]
   ```

In the following questions, we will explore the performance of learned decision trees on validation data and introduce the problem of "overfitting". **All questions below are asked on the `heart` dataset (`heart_train.tsv` and `heart_val.tsv`).**

4. **[15 Points]** Iterate the maximum depth limitation in the range $[0, 8]$, collect the training accuracy and validation accuracy. Plot both metrics with respect to the maximum depth in a single figure. Label your axes as well as your training and validation accuracy plots.

   Is the validation accuracy monotonically increasing as the maximum depth limit increases? What about the training accuracy? Report the maximum validation accuracy observed and the maximum depth limit that at which it is achieved.

The phenomenon you have (hopefully) witnessed, wherein the model struggles to extrapolate effectively from the training dataset to the validation dataset, is widely referred to as *overfitting*. One approach to mitigating the overfitting dilemma involves constraining the complexity of the classifier.

Constraining the maximum depth of the decision tree is perhaps the most straightforward method for addressing the issue of overfitting but there are many alternatives.

5. **[10 Points]** Among these alternatives is the imposition of a restriction on the size of splitting nodes. Specifically, if a node comprises *fewer than* M data points during the training process, further splitting is prohibitted.

   Train a decision tree under different minimum splitting sizes, with M ranging from 0 to the number of data points in the `heart` dataset: what is the optimal minimum splitting size, and what is the validation accuracy under this value?

   Note: If there are multiple minimum splitting sizes that achieve the optimal validation accuracy, report the smallest of them.

6. **[10 Points]** Another technique for mitigating overfitting involves changing the mutual information threshold. In this approach, at each node, if the best possible mutual information from a subsequent split falls below a designated threshold τ, the node is not divided any further.

   Train a decision tree with mutual information thresholds of $\{0.00, 0.01, \cdots, 0.99, 1.00\}$: what is the optimal information gain threshold, and what is the validation accuracy under this value?

   Note: If there are multiple mutual information thresholds that achieve the optimal validation accuracy, report the smallest of them.

4.4 Submission Instructions

Programming Please submit the following files to the **Homework 1 Programming** assignment on Gradescope. ensure you have completed the following file(s) for submission.

`decision_tree.py`

When submitting your solution, make sure to select and upload the file(s) shown above. **Any other files will be deleted.** Ensure the files have the exact same spelling and letter casing as above.

Written Questions Make sure you have completed all questions from Written component (including the collaboration policy questions) in the template provided. When you have done so, please submit your document in **PDF format** to the corresponding assignment slot on Gradescope.

5 Collaboration Questions

1. (a) Did you receive any help whatsoever from anyone in solving this assignment?

   (b) If you answered 'yes', give full details (e.g. "Jane Doe explained to me what is asked in Question 3.4")

2. (a) Did you give any help whatsoever to anyone in solving this assignment?

   (b) If you answered 'yes', give full details (e.g. "I pointed Joe Smith to section 2.3 since he didn't know how to proceed with Question 2")

3. (a) Did you find or come across code that implements any part of this assignment?

   (b) If you answered 'yes', give full details (book & page, URL & location within the page, etc.).